

INVELIO

Agentic AI for Indonesian Stock Analysis

Event	Duration	Team Size	Data Source
Sectors Hackathon 2026	17 Days	3 People	Sectors API + MCP

An iOS SwiftUI app backed by a Python agentic backend with LangGraph scoring, AI-powered chatbot, and real-time sentiment alerts. All stock data flows from the Sectors REST API and Sectors MCP, making Sectors the irreplaceable core of the product.

Key constraints:

- **1,000 Sectors API credits total** — Use mock data during development; aggressive caching in production
- **17-day timeline** — Ship simple first, iterate. Working simple > broken complex
- **Sectors must be primary data source** — Removing Sectors data must break the app's core functionality

Team

Person	Name	Role	Primary Focus
A	Vira	Caching + AI Agents	Cache layer, Chatbot Agent, Alert Agent
B	Dewa	Backend + AI Agent	FastAPI routes, Scoring Agent, API integration
C	Surya	Database + Infra + Frontend	PostgreSQL/Supabase, Docker, SwiftUI app

1. Technology Stack

1.1 Backend

Component	Technology	Purpose
Framework	FastAPI (Python 3.12)	REST API server with async support, auto-generated OpenAPI docs
AI Orchestration	LangGraph	State machine for multi-step agent workflows (nodes + edges)
Chatbot LLM	OpenAI / Gemini API	Chat response generation, question classification
Scoring LLM	OpenAI / Gemini API	Stock analysis reasoning, buy/hold/sell recommendations
Data Source	Sectors REST API	All stock data: prices, fundamentals, sectors, news, market overview
Agent Tools	Sectors MCP	Chatbot tool-calling — LLM autonomously picks which endpoints to query
Database	PostgreSQL (Supabase)	Scores, chat history, portfolio, alerts, API response cache
Caching	In-memory + PostgreSQL	TTL-based cache to minimize Sectors API credit usage

1.2 iOS Frontend

Component	Technology	Purpose
UI Framework	SwiftUI (iOS 17+)	Declarative UI with UINavigationController, TabView
Charts	Swift Charts	Stock price charts, portfolio P&L; visualization
Networking	URLSession async/await	API client with SSE streaming for chat
Project Gen	XcodeGen	Generate .xcodeproj from YAML — avoids merge conflicts

1.3 Infrastructure

Component	Technology	Purpose
Database Host	Supabase	Managed PostgreSQL with auth, real-time subscriptions, free tier

Component	Technology	Purpose
Containerization	Docker + Compose	Reproducible backend environment for all team members
CI/CD	GitHub Actions	Auto lint/test on PR (backend: ruff+pytest, iOS: xcodebuild)
Version Control	Git + GitHub	Feature branches, PR reviews, monorepo structure

2. System Architecture

2.1 Data Flow Overview

The system follows a layered architecture. Data flows from Sectors API through a caching layer into the AI agents, which process and store results in PostgreSQL. The FastAPI backend exposes these results as REST endpoints consumed by the iOS app.

Layer	Components	Data Direction
1. External APIs	Sectors REST API, Sectors MCP Server, OpenAI/Gemini API	Inbound: stock data, LLM responses
2. Cache Layer	In-memory dict + PostgreSQL sectors_cache table	Intercepts: saves API responses, serves cached data
3. AI Agents	Scoring Agent, Chatbot Agent, Alert Agent (LangGraph)	Process: reads cached data, calls LLM, writes results
4. FastAPI Backend	REST routes: /stocks, /stock/{ticker}, /chat, /alerts	Outbound: serves processed data as JSON + SSE
5. iOS App	SwiftUI views: Home, StockDetail, Chat, Portfolio, Alerts	Consumes: displays data from backend endpoints

2.2 Sectors API Integration Map

Sectors API is the single source of truth for all stock data. Every endpoint maps to specific app features and agents. Removing Sectors would break the entire product.

Sectors Endpoint	Data Provided	Used By	Cache TTL
/companies/report/{ticker}	Financials, ratios, valuation	Scoring Agent, Chatbot	24 hours
/daily/{ticker}	OHLCV price history (90 days)	Home, StockDetail, Alert	5 min
/companies/	All IDX tickers + sector classification	Scoring Agent	24 hours
/sector/report/	Sector performance, rotation	Scoring Agent	1 hour
/most-traded/	Most actively traded stocks	Home screen	5 min
/top-companies/	Top gainers/losers	Alert Agent, Home	5 min
/idx-total/	Composite index, market stats	Scoring Agent, Home	10 min
/news/	News articles, filterable by ticker	Chatbot, Alert Agent	15 min
/news/filings	Official filings + sentiment tags	Scoring Agent	1 hour

3. AI Agent Architecture

All three agents are built with LangGraph — a state machine framework where each agent is a graph of nodes (functions) connected by edges (transitions). The LLM (OpenAI/Gemini) is used within specific nodes for reasoning, not as the orchestrator.

3.1 Scoring Agent

Owner: Person B (Dewa) **Data access:** Sectors REST API (deterministic)

The Scoring Agent analyzes stocks using five data dimensions from Sectors API, computes weighted scores, and uses an LLM to generate a human-readable buy/hold/sell recommendation.

Node	Input	Process	Output
1. <code>fetch_market_data</code>	Ticker list	Calls Sectors API for each data dimension: • Fundamental: <code>/companies/report/{ticker}</code> → PBV, ROE, PE, DER • Macro: <code>/idx-total/</code> → IHSG composite index • Sector: <code>/sector/report/</code> → sector performance • Risk: <code>/daily/{ticker}</code> → volatility from price history • Sentiment: <code>/news/</code> + <code>/news/filings</code> → sentiment tags	Raw data dict per ticker
2. <code>compute_scores</code>	Raw data dict	Calculate weighted scores: • Fundamental score (30%): PBV, ROE, PE, DER • Macro score (15%): IHSG trend • Sector score (20%): sector vs market • Risk score (15%): volatility, drawdown • Sentiment score (20%): news sentiment ratio	Numerical scores 0–100
3. <code>llm_reasoning</code>	Scores + raw data	Send to OpenAI/Gemini with prompt: "Given these scores and data, generate a buy/hold/sell recommendation with reasoning in 2–3 sentences."	Recommendation + reasoning text

3.2 Chatbot Agent

Owner: Person A (Vira) **Data access:** Sectors MCP (LLM picks tools)

The Chatbot Agent uses Sectors MCP for tool-calling. The LLM autonomously decides which Sectors endpoints to query based on the user's natural language question. This is the "agentic" behavior that qualifies the project for the hackathon.

Node	Input	Process	Output
1. <code>classify_question</code>	User message	LLM categorizes: single stock, comparison, sector, market-wide, or general finance question	Question type + entities (tickers)
2. <code>retrieve_context</code>	Question type	LLM uses Sectors MCP tools — it decides which endpoints to call. Example: "Compare BBKA and BMRI" → calls <code>get_company_report</code> twice	Structured data from Sectors
3. <code>generate_response</code>	Context + question	LLM writes answer grounded in Sectors data. Quality check verifies no hallucination.	Streamed response (SSE)

3.3 Alert Agent

Owner: Person A (Vira) **Data access:** Sectors REST API (periodic polling)

Node	Input	Process	Output
1. fetch_market_data	Scheduled trigger	Calls /top-companies/ for price movers and /news/ for sentiment-tagged articles	Price changes + news data
2. detect_anomalies	Price + sentiment	Check thresholds: price change >5%, unusual volume, negative sentiment spike. Cross-reference price move with news.	Anomaly flags
3. generate_alerts	Anomaly flags	Create alert records with severity (high/medium/low), human-readable message, and source data link	Alert records → PostgreSQL

MCP vs REST in this project: The Scoring and Alert agents use Sectors REST API directly — your code decides which endpoints to call (deterministic). The Chatbot agent uses Sectors MCP — the LLM decides which endpoints to call based on the user's question (agentic). Both approaches qualify for the hackathon; MCP demonstrates true agentic tool-use.

4. Database Schema (PostgreSQL / Supabase)

Owner: Person C (Surya)

stocks — Cached stock metadata from Sectors API

Column	Type	Description
ticker	VARCHAR(10) PK	e.g. BBKA
name	VARCHAR(200)	Company name
sector	VARCHAR(100)	Sector classification
subsector	VARCHAR(100)	Subsector
updated_at	TIMESTAMP	Last refresh from Sectors

scores — AI-generated stock scores and recommendations

Column	Type	Description
id	SERIAL PK	Auto-increment
ticker	VARCHAR(10) FK	References stocks
fundamental_score	FLOAT	PBV, ROE, PE, DER analysis (0–100)
macro_score	FLOAT	IHSG trend analysis (0–100)
sector_score	FLOAT	Sector vs market (0–100)
risk_score	FLOAT	Volatility, drawdown (0–100)
sentiment_score	FLOAT	News sentiment ratio (0–100)
overall_score	FLOAT	Weighted composite (0–100)
recommendation	VARCHAR(10)	BUY / HOLD / SELL
reasoning	TEXT	LLM-generated explanation
scored_at	TIMESTAMP	When the agent ran

chat_history — Conversation logs for the chatbot

Column	Type	Description
id	SERIAL PK	Auto-increment
session_id	UUID	Groups messages by session
role	VARCHAR(20)	user / assistant

Column	Type	Description
content	TEXT	Message text
created_at	TIMESTAMP	Message timestamp

portfolio — User's stock holdings

Column	Type	Description
id	SERIAL PK	Auto-increment
ticker	VARCHAR(10) FK	References stocks
shares	INTEGER	Number of shares held
buy_price	FLOAT	Average purchase price
added_at	TIMESTAMP	When added to portfolio

alerts — Agent-generated notifications

Column	Type	Description
id	SERIAL PK	Auto-increment
ticker	VARCHAR(10) FK	References stocks
alert_type	VARCHAR(30)	price_spike / volume_surge / sentiment_shift
severity	VARCHAR(10)	high / medium / low
message	TEXT	Human-readable alert message
is_read	BOOLEAN	Default false
created_at	TIMESTAMP	When generated

sectors_cache — Cached Sectors API responses (credit-saving)

Column	Type	Description
cache_key	VARCHAR(200) PK	e.g. company_report:BBCA
data	JSONB	Raw API response
ttl	INTEGER	TTL in seconds
cached_at	TIMESTAMP	When stored

5. Caching Strategy

Owner: Person A (Vira)

CRITICAL: With only 1,000 Sectors API credits for the entire project (development + testing + demo), caching is essential. Without it, the team could exhaust credits by Day 10. With caching, estimated usage drops from ~2,000 calls to ~290 calls (85% reduction).

5.1 Two-Layer Cache Design

Layer 1 — In-memory (primary): Python dictionary with TTL tracking. Instant reads, zero latency. Lost on server restart.

Layer 2 — PostgreSQL (persistent): The sectors_cache table. Survives server restarts. Shared across multiple backend workers.

Lookup order: In-memory → PostgreSQL → Sectors API. Each cache hit avoids spending a credit.

5.2 TTL Configuration

Data Type	Endpoints	TTL	Reason
Prices / Trending	/daily, /most-traded, /top-companies	5 min	Prices move fast; 5 min staleness is acceptable
Market Index	/idx-total	10 min	Composite moves slower than individual stocks
News	/news	15 min	News is time-sensitive but not per-second
Sector Reports	/sector/report, /news/filings	1 hour	Sector rotation is a daily-level view
Fundamentals	/companies/report, /companies	24 hours	PE, ROE, revenue change quarterly

5.3 Credit Budget Allocation

Usage	Without Cache	With Cache	Savings
Home screen loads	~500	~50	90%
Stock detail views	~800	~80	90%
Scoring agent runs	~300	~60	80%
Chatbot testing	~400	~100	75%
TOTAL	~2,000	~290	85%

5.4 Development Strategy: Mock Data First

During development, use mock/hardcoded JSON responses that mirror the Sectors API format. This means zero credits spent while building and testing UI, routes, and agent logic. Switch to real API calls only for integration testing and the final demo.

- **Person C (Surya):** Build SwiftUI screens with hardcoded JSON — no backend needed
- **Person B (Dewa):** Test FastAPI routes with mock Sectors responses
- **Person A (Vira):** Develop agent logic with saved sample API responses

6. 17-Day Development Timeline

Phase 1: Foundation & Sectors Integration (Days 1–3)

Goal: Backend serves real Sectors data; iOS skeleton connects to it

Task	Owner	Day
Project repo setup, CI, folder structure	ALL	1
PostgreSQL schema design + Supabase setup	C (Surya)	1–2
Sectors API client wrapper (auth, rate limiting, response models)	B (Dewa)	1–2
Caching layer (in-memory + PostgreSQL, TTL logic)	A (Vira)	1–2
Mock data fixtures (sample Sectors API responses as JSON files)	A (Vira)	2
FastAPI skeleton with /api/status, CORS, config	B (Dewa)	2
iOS project setup, SwiftUI tab structure, API client	C (Surya)	1–3
REST endpoints: /stocks, /stock/{ticker}, /market-overview	B (Dewa)	3
Docker setup for backend	C (Surva)	3

Day 3 Checkpoint: Sectors API returns data → backend caches it → iOS displays a stock list with real tickers and prices. Prove the pipeline works end-to-end.

Phase 2: Scoring Agent & Home Screen (Days 4–7)

Goal: AI-powered stock scoring works end-to-end; Home screen shows live data

Task	Owner	Day
Scoring Agent — fetch_market_data node (5 data dimensions)	B (Dewa)	4–5
Scoring Agent — compute_scores node (weighted formula)	B (Dewa)	5–6
Scoring Agent — llm_reasoning node (buy/hold/sell + explanation)	B (Dewa)	6–7
HomeView — market overview, trending stocks, AI recommendations	C (Surya)	4–6
StockDetailView — price chart, fundamentals, AI insight card	C (Surya)	6–7
Scoring endpoint tests + data pipeline validation	A (Vira)	4–7
Begin Chatbot Agent research — Sectors MCP integration	A (Vira)	6–7

Day 7 Checkpoint: Scoring Agent runs on real Sectors data and produces a ranked top-10 list with LLM-generated reasoning. This is the hackathon's core differentiator.

Phase 3: Chatbot Agent & Portfolio (Days 8–11)

Goal: Conversational AI answers stock questions; Portfolio tracking works

Task	Owner	Day
Chatbot Agent — Sectors MCP setup + tool definitions	A (Vira)	8
Chatbot Agent — classify_question + retrieve_context nodes	A (Vira)	8–9
Chatbot Agent — generate_response + quality_check nodes	A (Vira)	9–10
Streaming chat endpoint (/api/chat with SSE)	B (Dewa)	10
ChatbotView — message bubbles, streaming, suggested prompts	C (Surya)	8–10
PortfolioView — holdings, P&L; chart, per-stock breakdown	C (Surya)	10–11
Chat history persistence + portfolio data model	B (Dewa)	8–11

Day 11 Checkpoint: User can ask the chatbot "How is BBKA performing?" and get a contextual answer grounded in Sectors data. Streaming response works on iOS.

Phase 4: Alert Agent & Integration (Days 12–14)

Goal: All three agents run together; notifications work; full feature set

Task	Owner	Day
Alert Agent — monitors Sectors data for significant price + sentiment moves	A (Vira)	12–13
Alert endpoint + notification integration	B (Dewa)	13
NotificationView — alert feed, severity indicators	C (Surya)	12–13
End-to-end integration testing (all agents + iOS)	ALL	12–14
Bug bash — fix data flow issues between agents	ALL	14

Day 14 Checkpoint: All three agents running. Home, Portfolio, Chatbot, and Notifications screens functional. No new features after this — only polish and fixes.

Phase 5: Polish, Demo & Submission (Days 15–17)

Goal: Product looks and works great for judging; documentation complete

Task	Owner	Day
UI polish — animations, loading states, empty states, error handling	C (Surya)	15–16
App icon, launch screen, onboarding flow	C (Surya)	15
Backend hardening — error handling, rate limit fallbacks	B (Dewa)	15–16
Cache optimization — ensure credits survive demo day	A (Vira)	15

Task	Owner	Day
README, architecture diagram, setup guide	ALL	15–16
Record demo video — full workflow walkthrough	C (Surya)	16–17
Final testing on clean environment	ALL	17
Submit — public repo + video	ALL	17

7. Project Structure

Sectors-Hackathon/

■ ■ ■ ■ .github/workflows/

■ ■ ■ ■ backend.yml # Python CI: ruff + pytest

■ ■ ■ ■ ios.yml # iOS CI: xcodebuild

■ ■ ■ ■ backend/

■ ■ ■ ■ app/

■ ■ ■ ■ api/routes/ # FastAPI endpoint handlers

■ ■ ■ ■ agents/ # LangGraph agents (scoring, chatbot, alert)

■ ■ ■ ■ clients/ # External API clients (Sectors, LLM)

■ ■ ■ ■ db/ # Database setup, migrations

■ ■ ■ ■ models/ # Pydantic schemas

■ ■ ■ ■ tests/ # Backend unit + integration tests

■ ■ ■ ■ Dockerfile

■ ■ ■ ■ docker-compose.yml

■ ■ ■ ■ pyproject.toml

■ ■ ■ ■ requirements.txt

■ ■ ■ ■ .env.example

■ ■ ■ ■ ios/Invelio/

■ ■ ■ ■ Sources/

■ ■ ■ ■ App/ # App entry point, ContentView (tabs)

■ ■ ■ ■ Views/ # SwiftUI views per screen

■ ■ ■ ■ Models/ # Data models (Codable structs)

■ ■ ■ ■ Services/ # APIClient, networking

■ ■ ■ ■ Components/ # Reusable UI components

■ ■ ■ ■ Tests/

■ ■ ■ ■ project.yml # XcodeGen spec

■ ■ ■ ■ .gitignore

■ ■ ■ ■ .editorconfig

■ ■ ■ ■ README.md

8. Risks & Mitigations

Risk	Severity	Mitigation
Sectors API credits run out	HIGH	Caching (85% reduction), mock data during dev, budget tracking. Reserve 300 credits for demo.
Sectors API rate limits or downtime	HIGH	Cache layer handles repeated requests. Mock data layer ensures frontend never blocks on API issues.
LLM API costs spiral during dev	MEDIUM	Use cheapest model tier for dev/testing. Cache LLM responses for identical queries. Set budget alerts.
iOS ↔ Backend integration delays	MEDIUM	Define API contract (response shapes) on Day 2. Frontend uses mock data until endpoints are live.
Scope creep in agent complexity	MEDIUM	Ship simple scoring first (Day 7). Only add self-check loops if time permits. Working simple > broken complex.
Team member blocked on dependency	LOW	Nobody waits. Use mocks/stubs/hardcoded data. Connect real data when ready.

9. Collaboration Workflow

9.1 Git Workflow

- **main** branch is always deployable. Never push directly to main.
- Create **feature branches** per task: feature/caching-layer, feature/scoring-agent, feature/home-view
- Open a **Pull Request** when done. CI runs automatically (lint + test).
- Get at least **one review** from a teammate before merging.
- **Conflicts are rare** — Person C works in ios/, Persons A+B work in backend/.

9.2 Communication Protocol

The critical handoff is the **API contract** — what the backend returns and what the iOS app expects.

1. Person B finishes an endpoint → shares URL + response JSON shape with Person C
2. Person C updates APIClient.swift and connects the screen to real data
3. Person A writes tests to ensure the endpoint doesn't regress

9.3 Daily Standup (5 minutes)

Each person answers three questions:

1. What did I finish?
2. What am I doing today?
3. Do I need anything from someone else?

9.4 When Someone is Blocked

Situation	Solution
Person C needs an endpoint that doesn't exist yet	Use mock/hardcoded JSON in Swift, swap when endpoint is live
Person B needs the DB schema that C hasn't built yet	Use a simple dict or in-memory store, swap later
Person A needs to test an agent that B hasn't finished	Write the test with expected input/output, mark it @pytest.mark.skip

The rule: Nobody waits. Use mocks, stubs, or hardcoded data to keep moving. Connect the real thing when it's ready. A team that ships beats a team that coordinates perfectly.

10. Hackathon Qualification Checklist

Requirement	How Invelio Meets It
Multi-step reasoning flows	Scoring Agent: fetch data → compute scores → LLM reasoning → recommendation
Custom tool-use pipelines	Chatbot Agent uses Sectors MCP tools, orchestrated by custom LangGraph code
Routing between data sources	Agent decides: cached data vs. fresh API call vs. sector report vs. company report
Memory or state management	Chat history, portfolio state, cached scores in PostgreSQL
Autonomous task execution	Alert Agent monitors data and generates alerts without user prompting
Purpose-built interface	Custom iOS SwiftUI app — not a generic chat client or off-the-shelf tool
Sectors as primary data source	All stock data flows from Sectors API. Removing Sectors breaks scoring, chatbot, and alerts.